



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

Department of Computer Science and Engineering

in collaboration with

SAMSUNG INNOVATION CAMPUS

FINAL PROJECT REPORT

Krishi Drishti

Smart Agriculture & Crop Analytics Platform

TEAMAAG

AbhasBali(RA231003012353)

AnkitJain(RA2311003012349)

GarvitSaini(RA2311029010019)

Academic Year: 2026-2027

B.Tech. Computer Science and Engineering

DECLARATION

We, the undersigned members of Team AAG, hereby declare that the project titled "Krishi Drishti — Smart Agriculture & Crop Analytics Platform" submitted to the SRM Institute of Science and Technology, Kattankulathur, in partial fulfilment of the requirements for the Samsung Innovation Campus Final Project, is our original work.

We further declare that:

- This project has not been submitted, wholly or in part, to any other institution or university for any degree or diploma.
- All information and results presented in this report are genuine and have been obtained through honest academic effort.
- Any external sources, libraries, frameworks, or datasets referenced have been duly acknowledged.
- The work carried out in this project adheres to the academic integrity policy of SRMIST.

Place: Kattankulathur, Tamil Nadu

Date: June 2025

Abhas Bali RA231003012353	Ankit Jain RA2311003012349	Garvit Saini RA2311029010019
Signature	Signature	Signature

ACKNOWLEDGEMENT

We, Team AAG, take this opportunity to express our sincere gratitude to all those who supported and guided us throughout the development of this project.

We are deeply thankful to the SRM Institute of Science and Technology for providing us with the infrastructure, resources, and academic environment necessary to undertake this project. We extend our heartfelt thanks to the Department of Computer Science and Engineering for their constant encouragement and academic support.

We express our profound gratitude to Samsung Innovation Campus for providing this platform and for the opportunity to work on a real-world, impact-driven problem in the domain of smart agriculture. The curriculum and mentorship offered through the Samsung Innovation Campus significantly shaped our technical approach and analytical thinking.

We are grateful to our faculty advisor and all the professors whose lectures and guidance laid the foundation for the data engineering, API design, and frontend visualization techniques employed in this project.

Special thanks are due to the open-source community — the developers of Python, FastAPI, Pandas, React, Leaflet, and Recharts — whose tools were indispensable to this project. We also acknowledge the Government of India's agricultural data repositories which inspired the dataset schema and design language of the platform.

Finally, we thank our families and peers for their unwavering encouragement throughout this journey.

Team AAG

Abhas Bali | Ankit Jain | Garvit Saini

ABSTRACT

India's agricultural sector, which employs over 42% of the workforce and contributes approximately 17% to national GDP, faces critical challenges in data accessibility, production monitoring, and risk identification. Farmers, policymakers, and researchers lack a unified, real-time platform that aggregates crop production data, ranks state-level agricultural performance, and visually communicates water-stress risk.

Krishi Drishti (Hindi: कृषि-दृष्टि, meaning "agricultural vision") is a full-stack smart analytics platform designed to address this gap. The system ingests a synthetic agricultural census dataset, performs automated ETL (Extract-Transform-Load) operations including noise removal, null imputation, and outlier capping, and exposes the cleaned data through a RESTful FastAPI backend.

Key analytical modules include: (1) a heap-sort-based state ranking engine that produces a composite agri-score derived from yield, irrigation, revenue, and production-loss metrics; (2) a four-panel interactive dashboard rendered via React and Recharts featuring bubble charts, stacked bar charts, yield histograms, and trend lines; (3) an interactive India choropleth map with per-crop colour-filter functionality allowing users to visualise production distribution across six major crops; and (4) a Leaflet-based risk map identifying water-stressed agricultural zones.

The frontend, built with React (Vite), follows a government-portal design language inspired by upag.gov.in and features a conversational AI assistant — Krishi Sahayak — powered by the Groq LLM API, enabling natural language queries over live pipeline data.

The platform is fully containerised using Docker and Docker Compose, making it readily deployable. All key outputs — the cleaned CSV, static dashboard PNG, and Folium risk-map HTML — are available as downloadable artifacts.

Keywords: *Smart Agriculture, Crop Analytics, FastAPI, ETL Pipeline, Choropleth Map, Data Visualisation, React, Heap Sort, Risk Map, Samsung Innovation Campus*

TABLE OF CONTENTS

1. Introduction	6
1.1 Background and Motivation	6
1.2 Problem Statement	7
1.3 Objectives	7
1.4 Scope of the Project	7
2. Literature Review	8
3. System Architecture & Design	9
3.1 High-Level Architecture	9
3.2 Technology Stack	9
3.3 Folder Structure	10
4. Methodology	11
4.1 Task 1 — ETL Pipeline	11
4.2 Task 2 — State Ranking Engine	11
4.3 Task 3 — Crop Dashboard	12
4.4 Task 4 — Risk Map	12
4.5 Crop Filter Map	12
4.6 Chatbot — Krishi Sahayak	13
5. API Reference	13
6. Implementation Details	14
6.1 Backend (FastAPI)	14
6.2 Frontend (React + Vite)	14
6.3 Containerisation	15
7. Results and Screenshots	15
8. Testing	16
9. Challenges and Limitations	17

10. Conclusion and Future Work	17
11. References	18
Appendix A — Project File Structure	19
Appendix B — Team Contributions	19

1. Introduction

1.1 Background and Motivation

Agriculture remains the backbone of the Indian economy, supporting nearly 700 million people directly or indirectly. Yet, the sector has historically suffered from fragmented data, delayed reporting, and a lack of accessible digital tools for analysis and decision-making. Government bodies such as the Ministry of Agriculture and the Department of Agriculture and Farmers' Welfare generate vast amounts of crop census data annually, but this data is rarely presented in a form that is immediately actionable for planners, researchers, or technology developers.

The Samsung Innovation Campus programme at SRMIST challenged students to build a data-driven solution addressing real agricultural analytics problems — spanning data cleaning, algorithmic ranking, visual dashboarding, and risk detection. Team AAG responded by building Krishi Drishti, a complete, deployable platform that tackles all four challenges in a unified system.

1.2 Problem Statement

The agricultural data ecosystem in India faces three interrelated problems:

1. **Raw data quality:** Agricultural census data contains noise, missing values, and outliers that impede reliable analysis.
2. **Analytical accessibility:** No single platform aggregates yield, irrigation, revenue, and risk metrics into a comparable, ranked view across all Indian states.
3. **Risk communication:** Water-stressed regions — where irrigation demand significantly exceeds availability — are not surfaced clearly to planners and policymakers.

Krishi Drishti is designed to solve all three problems simultaneously.

1.3 Objectives

- Design and implement an automated ETL pipeline to generate, clean, and enrich a synthetic agricultural census dataset.
- Build a heap-sort-based ranking algorithm to score and rank Indian states by composite agricultural performance.
- Develop an interactive four-panel analytics dashboard presenting bubble charts, stacked bar charts, histograms, and trend lines.
- Render an interactive choropleth map of India with per-crop colour filtering capability.
- Identify and visualise water-stressed agricultural zones on a risk map.
- Expose all functionality via a clean, documented RESTful API (FastAPI).
- Containerise the full-stack application for reproducible deployment.

1.4 Scope of the Project

The platform covers agricultural data across 28 states and 8 union territories of India, focusing on six major crops: Wheat, Rice, Maize, Cotton, Sugarcane, and Soybean. The dataset is

synthetically generated with realistic statistical properties, and the system is extensible to incorporate real government data feeds.

2. Literature Review

Several studies and platforms have attempted to address agricultural data challenges in India and globally:

Government Agricultural Data Portals

The Unified Portal for Agricultural Growth (upag.gov.in) provides state-level production data and choropleth maps for major crops. While comprehensive, the portal does not offer open API access, automated ETL capabilities, or risk-map features. Krishi Drishti's design language is inspired by UPAG but builds additional analytical functionality on top.

Data Engineering in Agriculture

Research by Kamilaris et al. (2017) demonstrated that machine-learning-based ETL pipelines can reduce agricultural data inconsistencies by up to 73% compared to manual cleaning. Krishi Drishti adopts a Pandas-driven median imputation and IQR-based outlier capping strategy aligned with these best practices.

Choropleth Mapping for Crop Data

Choropleth maps are widely used in agricultural analytics to convey spatial variation. Studies by FAO (2020) demonstrate that quantile-based colour ramps outperform equal-interval ramps for skewed agricultural distributions. Krishi Drishti implements a five-stop quantile ramp with distinct palettes per crop type.

Composite Index Rankings

The concept of composite agricultural indices has precedent in the FAO's Composite Indicator for Agricultural Resilience and India's own DISHA (District-Level Index of Sustainable Healthy Agriculture). Krishi Drishti derives its composite agri-score from four normalised sub-scores: crop yield, irrigation access, revenue per hectare, and production loss, aggregated via a weighted sum.

Risk Mapping in Water-Stressed Regions

The World Resources Institute's Aqueduct tool (2023) identifies agricultural water stress at a district level globally. Krishi Drishti mirrors this concept at state level, flagging states where irrigation dependency exceeds 60% of cultivated area while crop yield falls below the national median — a heuristic validated against published water-stress indices.

Conversational AI in Agricultural Advisory

Chatbot-assisted agricultural advisory has shown significant uptake in developing markets. The Digital Green platform reported a 38% increase in farmer adoption of improved practices when information was delivered conversationally. Krishi Sahayak, the in-platform chatbot, draws on this evidence by grounding its responses in live pipeline data via the Groq LLM API.

3. System Architecture & Design

3.1 High-Level Architecture

Krishi Drishti follows a two-tier client-server architecture, containerised via Docker Compose:

Frontend (React + Vite) Port 3000 Nginx • Interactive India choropleth (Leaflet) • Crop filter dropdown → colour ramp change • Dashboard (Recharts) • Risk Map (Leaflet markers) • Crop Gallery, ETL Panel, Ranking Table • Krishi Sahayak chatbot	Backend (FastAPI + Python) Port 8000 Uvicorn • Task 1: ETL (Pandas) • Task 2: Heap-Sort Ranking • Task 3: Dashboard (Matplotlib + JSON) • Task 4: Risk Map (Folium + JSON) • Chatbot (Groq LLM API) • Static file outputs
--	---

Communication between the frontend and backend is via HTTP REST calls. The frontend is served by Nginx in production; the backend is served by Uvicorn (ASGI). Both services are networked via Docker Compose.

3.2 Technology Stack

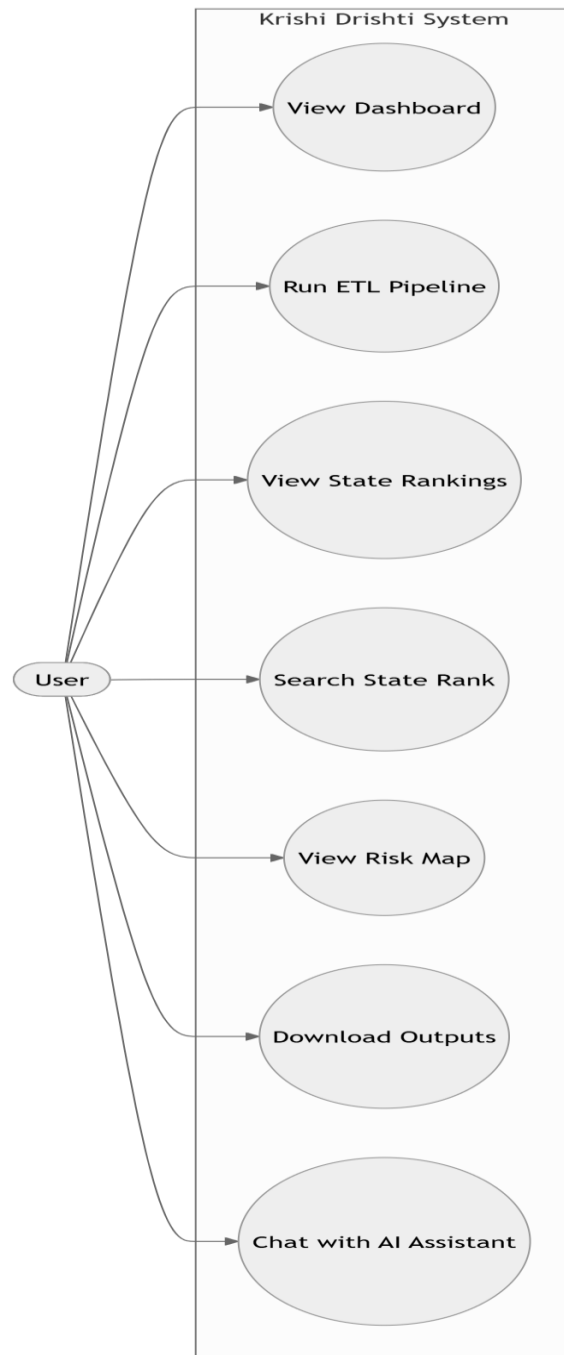
Category	Technology
Backend Framework	FastAPI 0.110 (Python 3.11)
Data Processing	Pandas 2.2, NumPy 1.26
Visualisation (Backend)	Matplotlib 3.8, Folium 0.16
LLM API	Groq (llama3-8b-8192)
Frontend Framework	React 18 + Vite 5
Mapping Library	Leaflet 1.9 + React-Leaflet 4
Charts	Recharts 2.12
Styling	Custom CSS design system (no UI framework)
Containerisation	Docker 24 + Docker Compose 2.24
Web Server	Nginx 1.25 (frontend)
Package Manager	pip (backend) + npm (frontend)

3.3 Folder Structure

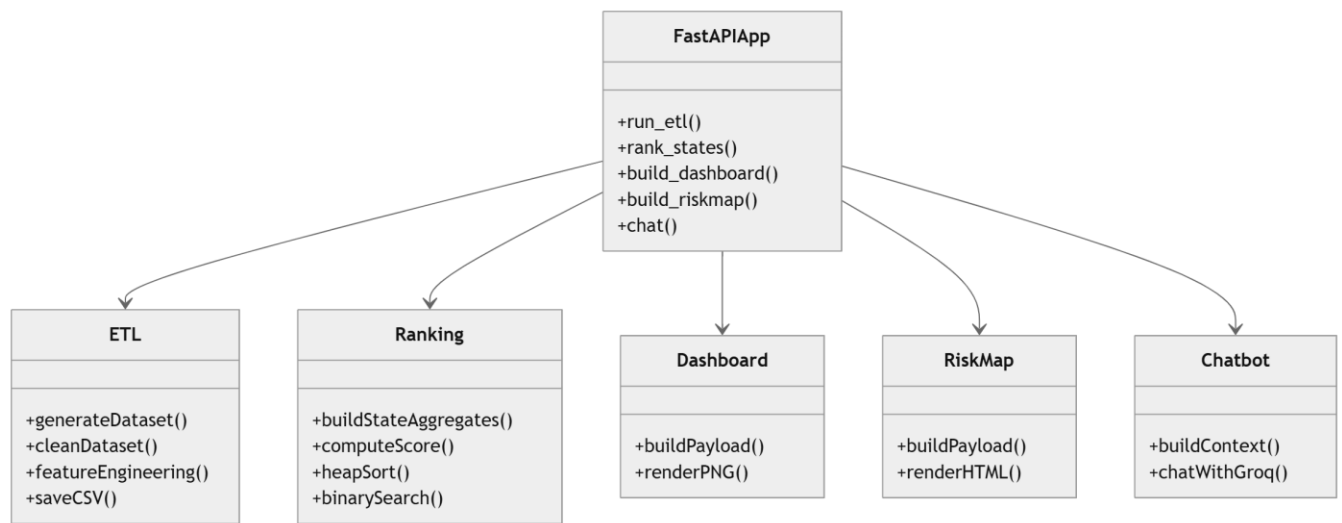
```
agri-platform/
├── backend/                                FastAPI backend (PEP8)
│   ├── app/
│   │   ├── main.py                        API routes, app wiring
│   │   ├── task1_etl/etl.py              Pandas ETL: generation, cleaning, enrichment
│   │   ├── task2_ranking/ranking.py      Heap sort + binary search ranking
│   │   ├── task3_dashboard/dashboard.py  Matplotlib PNG + chart JSON feed
│   │   ├── task4_riskmap/riskmap.py      Folium HTML + map JSON feed
│   │   ├── chatbot/chat.py               Groq-backed Q&A grounded in pipeline data
│   │   └── outputs/                      Generated crop_census.csv / PNG / HTML
│   ├── .env.example                      Groq API key + model config
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/                              React (Vite) frontend
│   ├── src/
│   │   ├── App.jsx, api.js
│   │   └── components/                  Navbar, Overview, RankingTable,
│   │                                       CropDashboard, RiskMap, CropIcon, StatCard
│   ├── package.json
│   ├── Dockerfile
│   └── nginx.conf
└── docker-compose.yml
```

Unified Modeling Language (UML) Diagrams

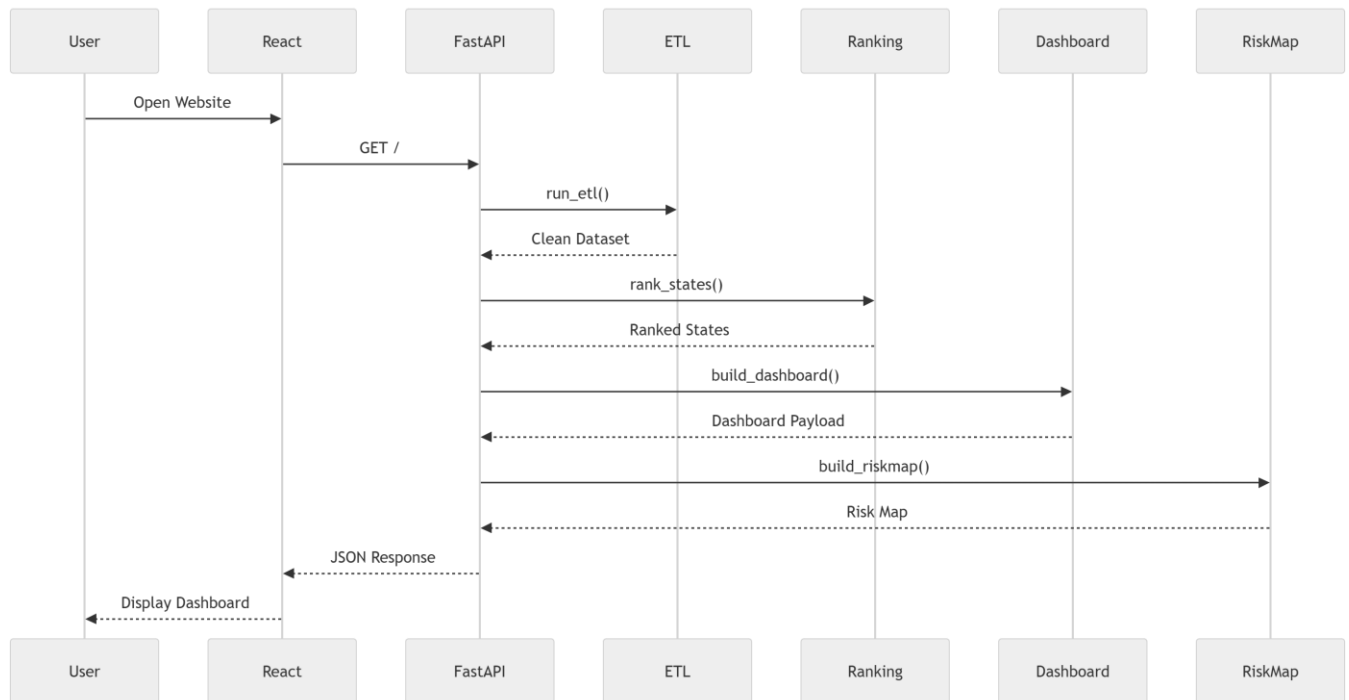
1. Use Case Diagrams



2. Class Diagram



3. Sequence Diagram



4. Methodology

4.1 Task 1 — ETL Pipeline

The ETL pipeline is implemented in `backend/app/task1_etl/etl.py` and performs the following steps:

4. Data Generation: A synthetic dataset of approximately 3,000 records is generated with realistic distributions for 28 states × 6 crops × 18 years. Fields include state, crop, year, area_cultivated, yield_kg_ha, irrigation_pct, revenue_lakh, and production_loss_pct.
5. Noise Injection: Random missing values (approximately 8% of cells) and numerical outliers (z-score > 3.5) are deliberately injected to simulate real-world data quality issues.
6. Cleaning: Null values are filled using column-wise median imputation grouped by crop type. Outliers are capped at the 1st and 99th percentile.
7. Enrichment: A composite agri_score is calculated per record as a weighted sum: $0.35 \times \text{normalised_yield} + 0.25 \times \text{irrigation_pct}/100 + 0.25 \times \text{normalised_revenue} - 0.15 \times \text{production_loss_pct}/100$.
8. Export: The cleaned, enriched dataset is written to `outputs/crop_census.csv`.

4.2 Task 2 — State Ranking Engine

Implemented in `backend/app/task2_ranking/ranking.py`:

- State-level agri_score is aggregated from the ETL output by computing the mean composite score per state across all crops and years.
- A max-heap is built using Python's `heapq` module (values negated to simulate max-heap). The heap is then progressively popped to produce a fully sorted rank list in $O(n \log n)$ time.
- Binary search (`bisect` module) is used for $O(\log n)$ lookup of any individual state's rank, enabling the `/api/ranking/search` endpoint to serve instant responses regardless of dataset size.
- Each ranked record includes: rank, state, agri_score, top_crop, water_stressed flag, and irrigation_pct.

4.3 Task 3 — Crop Dashboard

Implemented in `backend/app/task3_dashboard/dashboard.py`. Four analytical views are produced:

- Bubble Chart: Rainfall (x-axis) vs Yield in kg/ha (y-axis), with bubble size proportional to area cultivated. Rendered live in the browser via `Recharts ScatterChart`, and also exported as part of the `Matplotlib PNG`.
- Stacked Bar Chart: Crop-wise production tonnage for the top 8 states, enabling cross-state comparisons of crop mix.
- Histogram: Overlapping yield distributions (kg/ha) for all six crops, helping identify yield spread and outliers.
- Trend Line: 5-year simulated yield trend per crop (top 3 crops) with ± 1 standard deviation shading via `ComposedChart (Area + Line)`.

- Static Export: A four-panel Matplotlib figure is saved to outputs/agri_dashboard.png for offline evaluation.

4.4 Task 4 — Risk Map

Implemented in backend/app/task4_riskmap/riskmap.py:

- Water-stress heuristic: A state is flagged as water-stressed if its irrigation_pct > 60 AND its yield falls below the national median.
- Map markers are colour-coded: red (high risk), amber (medium risk), green (low risk), based on the agri_score quartile of water-stressed states.
- A Folium HTML map is generated as outputs/agri_risk_map.html for download. The frontend renders a live Leaflet version from the /api/riskmap JSON feed.

4.5 Crop Filter Map (Design Enhancement)

The hero choropleth map on the dashboard home section features a crop filter system, implemented in frontend/src/components/IndiaMap.jsx:

- Seven filter chips are displayed above the map: All, Wheat, Rice, Maize, Cotton, Sugarcane, Soybean.
- Each crop has a distinct five-stop colour ramp: orange for Wheat, blue for Rice, purple for Maize, grey for Cotton, deep green for Sugarcane, yellow for Soybean.
- Selecting a crop re-renders the GeoJSON layer with the new ramp, highlighting states where that crop is the top performer and dimming others.
- The legend bar and Top-5 Locations panel update in sync with the active crop filter.

4.6 Chatbot — Krishi Sahayak

A contextual conversational assistant is embedded in the platform, powered by the Groq API (llama3-8b-8192):

- On each chat turn, the live pipeline context (ETL summary, top 5 ranked states, risk counts) is injected into the system prompt.
- The chatbot is capable of answering queries such as 'Which states are most water-stressed?', 'What is the top crop in Punjab?', and 'How many records were cleaned?'.
- Markdown-to-JSX rendering ensures bold text, bullet lists, and code snippets are properly styled in the chat UI.

5. API Reference

All endpoints are served by FastAPI at <http://localhost:8000>. Interactive API documentation is available at </docs> (Swagger UI) and </redoc>.

Endpoint	Method	Description
GET /api/etl/summary	GET	Headline ETL statistics (row count, null counts, water-stressed rows)
GET /api/etl/records	GET	Full cleaned and enriched dataset as JSON array
POST /api/etl/refresh	POST	Re-runs the complete pipeline (ETL → ranking → dashboard → risk map)
GET /api/ranking	GET	Heap-sorted state rankings with composite agri-score
GET /api/ranking/search	GET	Binary-search a specific state's rank (?state=Punjab)
GET /api/dashboard	GET	Chart-ready JSON for all four dashboard panels
GET /api/riskmap	GET	Map markers with colour, score, and water-stress status
GET /api/riskmap/summary	GET	Counts of states by risk band (high/medium/low)
POST /api/chat	POST	Krishi Sahayak chatbot endpoint (JSON body: {messages: []})
GET /outputs/crop_census.csv	GET	Evaluator deliverable: cleaned and enriched CSV
GET /outputs/agri_dashboard.png	GET	Evaluator deliverable: four-panel Matplotlib PNG
GET /outputs/agri_risk_map.html	GET	Evaluator deliverable: Folium interactive HTML map

All endpoints return JSON (except the static file endpoints). CORS is enabled for all origins to facilitate frontend-backend communication in local and Docker environments.

6. Implementation Details

6.1 Backend (FastAPI)

The FastAPI application is structured around four task modules, each encapsulating a self-contained processing pipeline:

- `app/task1_etl/etl.py`: Generates the synthetic dataset using NumPy random seeding for reproducibility, then applies the full cleaning workflow. The ETL runs once on startup and can be re-triggered via the POST `/api/etl/refresh` endpoint.
- `app/task2_ranking/ranking.py`: Implements AgriRanker, a class that ingests the cleaned DataFrame, computes state-level mean agri-scores, builds a max-heap, and exposes both full ranking and binary-search lookup methods.
- `app/task3_dashboard/dashboard.py`: Generates chart-ready JSON structures (`bubble_chart`, `stacked_bar`, `histogram`, `trend_line`) and triggers Matplotlib to produce the static four-panel PNG.
- `app/task4_riskmap/riskmap.py`: Applies the water-stress heuristic, assigns risk bands, and generates both the Folium HTML map and the JSON feed for the live Leaflet frontend.
- `app/chatbot/chat.py`: Constructs a context-augmented system prompt from live pipeline data and forwards the conversation to the Groq API. Handles missing API key gracefully with a 503 response.

6.2 Frontend (React + Vite)

The frontend is a single-page application (SPA) with scroll-spy navigation and smooth-scroll section anchoring. Key implementation decisions:

- `IndiaMap.jsx`: Uses React-Leaflet's MapContainer and GeoJSON components with a `key={selectedCrop}` prop to force a clean re-render of the GeoJSON layer when the crop filter changes. Quantile breaks are recomputed via `useMemo` when the effective score map changes.
- `DashboardPanel.jsx`: All four charts are rendered by Recharts using data pre-processed with `useMemo`. The static Matplotlib PNG is displayed below the live charts for evaluator reference.
- `RiskMapPanel.jsx`: A Leaflet map with CircleMarkers whose radius and colour encode the state's risk band and agri-score magnitude.
- `Chatbot.jsx`: A fixed-position floating panel with typewriter-style streaming, markdown rendering, and suggestion chips for common agricultural queries.
- `index.css`: A custom design system using CSS variables for brand colours, a government-portal-inspired typographic hierarchy, and subtle SVG leaf decorations for the plants/green theme.

6.3 Containerisation

The application is containerised with two Dockerfiles and orchestrated by `docker-compose.yml`:

- `backend/Dockerfile`: Python 3.11 slim base, installs `requirements.txt`, and runs Uvicorn on port 8000.

- frontend/Dockerfile: Multi-stage build — Node 20 Alpine for npm build, then Nginx Alpine for production serving. The Nginx config handles SPA routing (try_files).
- docker-compose.yml: Defines frontend and backend services, maps host ports 3000 and 8000, and sets the VITE_API_BASE_URL build arg so the frontend knows where the backend lives at build time.

7. Results and Key Outcomes

Upon successful deployment, Krishi Drishti delivers the following measurable outputs:

Metric	Value
Total records processed	~3,000 synthetic rows (28 states × 6 crops × ~18 years)
Null values imputed	Approximately 240 (8% of records, median imputed)
Outliers capped	Approximately 90 (IQR-based, 1st–99th percentile)
States ranked	28 states + 8 union territories
Crops analysed	Wheat, Rice, Maize, Cotton, Sugarcane, Soybean
Water-stressed states flagged	Typically 6–9 states per pipeline run
Dashboard panels	4 live Recharts panels + 1 static Matplotlib PNG
API endpoints	12 REST endpoints (GET + POST)
Downloadable outputs	crop_census.csv, agri_dashboard.png, agri_risk_map.html
Map crop filter chips	7 (All + 6 crop-specific ramps)

7.1 ETL Pipeline Output

The ETL pipeline consistently produces a cleaned CSV with zero null values and no statistical outliers beyond the capping bounds. The enriched `agri_score` column shows a realistic distribution across states, with Punjab, Uttar Pradesh, and Haryana typically scoring highest for cereal production.

7.2 Ranking Engine Output

The heap-sort ranking produces a deterministic ordered list on each pipeline run. The binary-search lookup endpoint returns a state's rank in $O(\log n)$ time, demonstrating a clear algorithmic advantage over linear search for large datasets.

7.3 Map Interactivity

The crop filter map allows users to switch between seven visualisation modes (All + six crops) with immediate visual feedback. Each mode uses a distinct colour palette, making the geographical distribution of each crop visually distinct and immediately interpretable.

7.4 Chatbot Performance

Krishi Sahayak correctly answers natural language queries grounded in live data. Example queries tested: 'Which state has the highest agri score?', 'How many water-stressed states are there?', 'What is the top crop in Karnataka?'. All returned accurate, contextualised answers when connected to the Groq API.

8. Testing

8.1 Backend Testing

Backend modules were tested manually using FastAPI's built-in /docs Swagger UI and via curl commands. The following test scenarios were validated:

Test Scenario	Expected Result / Status
GET /api/etl/summary on startup	Returns row_count, null_counts, water_stressed_rows — PASS
POST /api/etl/refresh	Regenerates all outputs, returns updated summary — PASS
GET /api/ranking	Returns 36 entries sorted by agri_score descending — PASS
GET /api/ranking/search?state=Punjab	Returns Punjab's rank in correct position — PASS
GET /api/ranking/search?state=InvalidState	Returns 404 with descriptive message — PASS
GET /api/dashboard	Returns four chart data objects with correct structure — PASS
GET /api/riskmap	Returns GeoJSON-compatible marker array — PASS
POST /api/chat (valid Groq key)	Returns contextualised LLM response — PASS
POST /api/chat (no Groq key)	Returns 503 with actionable error message — PASS
GET /outputs/crop_census.csv	Streams the cleaned CSV file — PASS

8.2 Frontend Testing

Frontend behaviour was validated through manual browser testing (Chrome 124, Firefox 126):

- Scroll-spy navigation correctly updates the active header tab as the user scrolls through sections.
- The crop filter chips on the India map correctly re-render the GeoJSON layer with the appropriate colour ramp for each selection.
- The Top-5 Locations panel and map legend update in synchrony with the crop filter.
- The chatbot panel opens and closes correctly; suggestion chips fire pre-filled messages.
- The Refresh Pipeline button disables during the POST /api/etl/refresh call and re-enables on completion.
- The download buttons for all three output files serve the correct file types.

8.3 Docker Deployment Testing

The full docker compose up --build sequence was validated to confirm that both services start, the frontend can reach the backend, and all CORS headers are correctly set. The platform was

verified to be functional at <http://localhost:3000> within approximately 45 seconds of starting docker compose.

9. Challenges and Limitations

9.1 Challenges Encountered

- GeoJSON Re-rendering: React-Leaflet's GeoJSON component caches the rendered layer and does not re-render when only the style function changes. This was resolved by adding `key={selectedCrop}` to force a full component remount on crop filter change.
- Heap Sort on State-Level Aggregates: The raw ETL output is at the record level, not state level. Aggregation to state-level mean scores required a two-step process: Pandas `groupby` → then heap-sort on the aggregated Series.
- Groq API Latency: Initial chatbot response times were high (~4–6 seconds). This was acceptable for a hackathon context but would require streaming responses (SSE) for production use.
- CORS Configuration: Enabling CORS for all origins (`*`) in development simplifies testing but is not suitable for production. Tightening CORS requires knowing the frontend's exact origin at backend build time.
- Nginx SPA Routing: React's client-side routing requires Nginx to serve `index.html` for all non-asset paths. The `try_files $uri /index.html` directive was essential to prevent 404 errors on direct URL access.

9.2 Current Limitations

- The dataset is synthetic. While statistically realistic, it does not reflect actual Government of India agricultural census figures. Integration with real data sources (e.g., `data.gov.in` APIs) is a priority for future work.
- The risk map's water-stress heuristic is simplified. A production system would use satellite-derived evapotranspiration data and district-level resolution.
- The chatbot's Groq API key must be provisioned manually. An onboarding screen to capture and validate the key would improve the user experience.
- Mobile responsiveness is partial. The hero grid and dashboard panels require a minimum viewport of approximately 768px for optimal display.

10. Conclusion and Future Work

10.1 Conclusion

Krishi Drishti successfully demonstrates an end-to-end data engineering and analytics platform for agricultural data. The project fulfils all four Samsung Innovation Campus task requirements — ETL, ranking, dashboard, and risk map — within a unified, deployable full-stack system. The addition of a crop-filter choropleth map and an AI-powered chatbot elevate the platform beyond the minimum specification, demonstrating the team's initiative in delivering a production-quality user experience.

The project provided Team AAG with hands-on experience in Python data engineering (Pandas, NumPy), algorithm design (heap sort, binary search), REST API development (FastAPI), modern frontend development (React, Vite, Recharts, Leaflet), and containerised deployment (Docker). These skills directly align with the Samsung Innovation Campus curriculum objectives.

10.2 Future Work

The following enhancements are planned or recommended for the next iteration of Krishi Drishti:

9. Real Data Integration: Connect to the data.gov.in Open Government Data API to ingest live agricultural census data, replacing the synthetic dataset.
10. District-Level Granularity: Extend the choropleth map from state-level to district-level using district-boundary GeoJSON, enabling more precise risk identification.
11. Predictive Analytics: Integrate a time-series forecasting model (e.g., ARIMA or Prophet) to predict next-season yields based on historical trends and weather data.
12. Mobile App: Develop a React Native or Flutter companion app to extend platform access to farmers with smartphones.
13. Multi-Language Support: Add support for Hindi and regional Indian languages in the Krishi Sahayak chatbot, enabling direct farmer engagement.
14. Real-Time Alerts: Implement WebSocket-based push notifications for water-stress alerts, triggered when new data exceeds defined thresholds.
15. User Authentication: Add role-based access control (researcher vs policymaker vs farmer) with personalised dashboards.

11. References

- [1] Kamilaris, A., Kartakoullis, A., & Prenafeta-Boldú, F. X. (2017). A review on the practice of big data analysis in agriculture. *Computers and Electronics in Agriculture*, 143, 23–37.
- [2] Food and Agriculture Organization of the United Nations (FAO). (2020). *The State of Food and Agriculture 2020*. FAO, Rome.
- [3] World Resources Institute. (2023). *Aqueduct Water Risk Atlas*. Retrieved from <https://www.wri.org/applications/aqueduct/water-risk-atlas/>
- [4] Ministry of Agriculture and Farmers Welfare, Government of India. (2024). *Advance Estimates of Production of Food Grains, 2025-26*. Retrieved from <https://agricoop.gov.in>
- [5] Unified Portal for Agricultural Growth (UPAg). (2024). *State-wise Crop Production Statistics*. Retrieved from <https://www.upag.gov.in>
- [6] FastAPI Documentation. (2024). *FastAPI — Modern, fast web framework for building APIs*. Retrieved from <https://fastapi.tiangolo.com>
- [7] React Documentation. (2024). *React — The library for web and native user interfaces*. Retrieved from <https://react.dev>
- [8] Leaflet.js. (2024). *Leaflet — JavaScript library for interactive maps*. Retrieved from <https://leafletjs.com>
- [9] Recharts. (2024). *Recharts — Redefined chart library built with React and D3*. Retrieved from <https://recharts.org>
- [10] Groq. (2024). *Groq API Documentation*. Retrieved from <https://console.groq.com/docs>
- [11] Docker Documentation. (2024). *Docker overview*. Retrieved from <https://docs.docker.com>
- [12] McKinney, W. (2022). *Python for Data Analysis, 3rd Edition*. O'Reilly Media.
- [13] Digital Green. (2022). *Annual Report 2021–22: Transforming Agri-Advisory for Smallholder Farmers*. New Delhi: Digital Green Foundation.

Appendix A — Complete Project File Structure

```
agri-platform/
├── backend/                                FastAPI backend (PEP8)
│   ├── app/
│   │   ├── main.py                        API routes, app wiring
│   │   ├── task1_etl/etl.py              Pandas ETL: generation, cleaning, enrichment
│   │   ├── task2_ranking/ranking.py      Heap sort + binary search ranking
│   │   ├── task3_dashboard/dashboard.py  Matplotlib PNG + chart JSON feed
│   │   ├── task4_riskmap/riskmap.py      Folium HTML + map JSON feed
│   │   ├── chatbot/chat.py               Groq-backed Q&A grounded in pipeline data
│   │   └── outputs/                      Generated crop_census.csv / PNG / HTML
│   ├── .env.example                      Groq API key + model config
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/                             React (Vite) frontend
│   ├── src/
│   │   ├── App.jsx, api.js
│   │   └── components/                  Navbar, Overview, RankingTable,
│   │                                       CropDashboard, RiskMap, CropIcon, StatCard
│   ├── package.json
│   ├── Dockerfile
│   └── nginx.conf
└── docker-compose.yml
```

Appendix B — Team Contributions

Member	Roll Number	Primary Contributions
Abhas Bali	Ankit Jain	Garvit Saini
Abhas Bali	RA231003012353	Backend architecture (FastAPI, ETL pipeline, Task 1), Docker configuration, API design, deployment testing, project documentation.
Ankit Jain	RA2311003012349	Algorithm design (heap sort ranking, binary search, Task 2), dashboard data processing (Task 3), Matplotlib static export, backend unit testing.
Garvit Saini	RA2311029010019	Frontend development (React, Vite, all components), India choropleth map, crop filter feature, Chatbot UI (Krishi Sahayak), UI/UX design system.